pontIA

MÁSTER INGENIERÍA DE DATOS

Consultas 2

Operaciones CRUD 2 - Recursos

Daniel Trabas

pontIA

Índice

Objetivo 3

Operaciones ACID 3

Operaciones CRUD 2: Consultas Agrupadas y ACID 3

PontIA

Objetivo

Este documento avanza sobre los fundamentos de las consultas para introducir la agregación de datos y las operaciones de escritura (creación, modificación y borrado de registros) en PostgreSQL, utilizando el esquema de reservas de vuelos bookings.

Operaciones ACID

Las propiedades ACID son las garantías que aseguran la validez de los datos ante errores de hardware o software:

- **Atomicidad (Atomicity):** garantiza que una transacción sea una unidad indivisible. O se confirman todos los cambios o no se aplica ninguno. Si una actualización de precio falla al final, el sistema revierte cualquier cambio previo de esa sesión.
- **Coherencia (Consistency):** asegura que cualquier transacción lleve a la base de datos de un estado válido a otro. En PostgreSQL, esto se aplica mediante constraints (restricciones) como UNIQUE, NOT NULL y llaves foráneas (FK), impidiendo datos huérfanos o duplicados.
- **Aislamiento (Isolation):** necesario para la concurrencia. Permite que múltiples transacciones ocurran simultáneamente sin interferir entre sí. Un usuario no verá datos parciales de una reserva que aún se está procesando.
- **Durabilidad (Durability):** una vez que el sistema confirma un "COMMIT", los datos persisten incluso ante fallos críticos de energía. Esto se logra mediante registros de transacciones que aseguran que el sistema vuelva al último estado conocido.

Operaciones CRUD 2: Consultas Agrupadas y ACID

SELECT con GROUP BY

La cláusula GROUP BY se utiliza para agrupar filas de una tabla que contienen los mismos valores en columnas específicas, permitiendo realizar cálculos sobre esos grupos. Según la documentación, "GROUP BY condensará en una sola fila todas las filas seleccionadas que compartan los mismos valores para las expresiones agrupadas" - según la documentación original de PostgreSQL.

Generalmente se utiliza junto con funciones de agregación como COUNT(), SUM(), AVG(), MAX() o MIN() que permiten obtener un resultado único a partir de un conjunto de registros de entrada.

Escenarios de uso:

- Contar cuántos vuelos hay por cada estado (programado, retrasado, cancelado).
- Sumar el importe total de las reservas realizadas en cada fecha.
- Calcular el precio medio de los segmentos de vuelo por clase de tarifa.

Ejemplos prácticos sobre demodb:

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

```sql
-- 1. Contar el número de vuelos agrupados por su estado actual
SELECT fl.status, COUNT(fl.flight_id) AS total_vuelos
FROM bookings.flights fl
GROUP BY fl.status;

-- 2. Calcular la recaudación total de reservas agrupada por fecha
SELECT bk.book_date, SUM(bk.total_amount) AS recaudacion_diaria
FROM bookings.bookings bk
GROUP BY bk.book_date
ORDER BY bk.book_date DESC;

-- 3. Calcular el precio medio de un segmento de vuelo según la clase de tarifa
SELECT sg.fare_conditions, AVG(sg.price) AS precio_medio
FROM bookings.segments sg
GROUP BY sg.fare_conditions;

-- 4. Contar cuántos asientos tiene cada modelo de avión combinando JOIN y
GROUP BY
SELECT ap.model, COUNT(st.seat_no) AS total_asientos
FROM bookings.airplanes ap
INNER JOIN bookings.seats st ON ap.airplane_code = st.airplane_code
GROUP BY ap.model;
```

Esquemas y Alias de Tabla

PostgreSQL organiza las tablas dentro de agrupaciones lógicas llamadas esquemas. Un esquema funciona como un espacio de nombres (namespace) donde residen las tablas; por defecto, se utiliza el esquema bookings. En el caso de nuestra base de datos llamada demodb, el esquema que utilizaremos será bookings.

Cuando la consulta incluye múltiples tablas, es una excelente práctica usar alias de tabla. Un alias es un nombre temporal y corto que le damos a una tabla en la cláusula FROM para simplificar la escritura de la consulta y evitar ambigüedades.

Limitadores de Resultados (Sin WHERE)

A menudo no queremos recuperar toda la tabla, sino sólo una porción. Podemos restringir el número de filas devueltas sin filtrar por condiciones utilizando modificadores al final de la consulta:

- **LIMIT:** Especifica el número máximo de filas a devolver.
- **OFFSET:** Indica cuántas filas saltar antes de empezar a devolver resultados. "Finalmente, las filas que caen fuera del rango especificado por LIMIT y OFFSET se descartan".
- **FETCH:** Funciona de manera similar a LIMIT, pero pertenece al estándar SQL moderno.

Ejemplos prácticos sobre demodb:

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

```sql
-- 1. Contar el número de vuelos agrupados por su estado actual
SELECT fl.status,
       COUNT(fl.flight_id) AS total_vuelos
FROM   bookings.flights fl
GROUP BY fl.status;

-- 2. Calcular la recaudación total de reservas agrupada por fecha
SELECT TO_CHAR(bk.book_date, 'YYYY-MM-DD') AS fecha,
       SUM(bk.total_amount) AS recaudacion_diaria
FROM   bookings.bookings bk
WHERE  bk.book_date BETWEEN '2026-01-01' AND '2026-01-31'
GROUP BY TO_CHAR(bk.book_date, 'YYYY-MM-DD')
ORDER BY TO_CHAR(bk.book_date, 'YYYY-MM-DD') DESC;

-- 3. Calcular el precio medio de un segmento de vuelo según la clase de tarifa
SELECT sg.fare_conditions,
       AVG(sg.price) AS precio_medio
FROM   bookings.segments sg
GROUP BY sg.fare_conditions;

-- 4. Contar cuántos asientos tiene cada modelo de avión combinando JOIN y
GROUP BY
SELECT ap.model,
       COUNT(st.seat_no) AS total_asientos
FROM   bookings.airplanes ap
INNER  JOIN bookings.seats st ON ap.airplane_code = st.airplane_code
GROUP BY ap.model;
```

INSERT

"La sentencia INSERT se utiliza para rellenar una tabla con filas". Es la operación que nos permite introducir nuevos datos en nuestra base de datos.

Escenarios de uso:

- **Insert sencillo:** registrar una única nueva reserva realizada por un cliente.
- **Insert de varios elementos:** generar múltiples billetes asociados a una misma reserva en una sola transacción.
- **Insert desde tablas/consultas:** mover o copiar registros masivos, como archivar vuelos cancelados en una tabla temporal.

Ejemplos prácticos sobre demodb:

pontIA

```sql
-- 1. Insert Sencillo: Crear una nueva reserva
INSERT INTO bookings.bookings (book_ref, book_date, total_amount)
VALUES ('XYZ123', '2026-08-01', 500.00);
-- Validamos que la reserva se ha insertado correctamente
SELECT *
FROM    bookings.bookings
WHERE   book_ref = 'XYZ123';

-- 2. Insert de Varios Elementos: Añadir dos billetes para la reserva anterior
INSERT INTO bookings.tickets (ticket_no, book_ref, passenger_id,
passenger_name, outbound)
VALUES ('1234567890123', 'XYZ123', '77788899A', 'John Doe', 'Y'),
       ('1234567890124', 'XYZ123', '11122233B', 'Jane Doe', 'Y');
-- Validamos que los billetes se han insertado correctamente
SELECT *
FROM    bookings.tickets
WHERE   book_ref = 'XYZ123';

-- 3. Insert desde tabla temporal: Guardar temporalmente reservas VIP (> 1000)
CREATE TEMPORARY TABLE temp_vip_bookings
AS
  SELECT 'T' as reg_type
         ,*
  FROM    bookings.bookings
  LIMIT 0;
-- Validamos que la tabla temporal se ha creado correctamente
SELECT *
FROM    temp_vip_bookings;
-- Insertar reservas VIP en la tabla temporal
INSERT INTO temp_vip_bookings
SELECT 'T',
       bk.*
FROM    bookings.bookings bk
WHERE   bk.total_amount >= 1000.00
LIMIT 1000;
 -- Validamos que las reservas VIP se han insertado correctamente en la tabla
temporal
SELECT *
FROM    temp_vip_bookings;

-- 4. Error de Inserción por violación de PK y su solución
-- Provocará error: "violates unique constraint" porque 'XYZ123' ya existe
INSERT INTO bookings.bookings (book_ref, book_date, total_amount)
VALUES ('XYZ123', '2026-08-02', 800.00);

-- Solución: Usar una PK nueva ('XYZ124') o usar ON CONFLICT (Upsert)
-- Realmente esta solución es similar a realizar un UPDATE si la PK ya existe,
pero en este caso, como queremos insertar una nueva reserva, usamos un book_ref
diferente.
INSERT INTO bookings.bookings (book_ref, book_date, total_amount)
VALUES ('XYZ123', '2026-08-02', 1200.00)
ON CONFLICT (book_ref) DO UPDATE SET total_amount = EXCLUDED.total_amount;
```

```sql
-- Validamos que la reserva se ha actualizado correctamente
SELECT *
FROM    bookings.bookings
WHERE   book_ref LIKE 'XYZ1%';

-- Solución: Usar una PK nueva ('ZZZZP9')
-- Buscamos el último book_ref para generar uno nuevo
SELECT MAX(book_ref) AS ultima_reserva
FROM    bookings.bookings
-- Generar un nuevo book_ref basado en la última reserva
INSERT INTO bookings.bookings (book_ref, book_date, total_amount)
VALUES ('ZZZZP9', '2026-08-02', 800.00);
-- Validamos que la nueva reserva se ha insertado correctamente
SELECT *
FROM    bookings.bookings
WHERE   book_ref LIKE 'ZZZZP%';
```

UPDATE

Cuando modificamos datos que ya se encuentran en las tablas de la base de datos, se denomina actualización.

La sentencia UPDATE permite cambiar los valores de una o varias columnas en las filas que cumplan una condición específica.

Escenarios de uso:

- Corregir un error ortográfico en el nombre de un pasajero.
- Aumentar el importe total de una reserva si el cliente añade servicios.
- Cambiar el estado de múltiples vuelos que han despegado simultáneamente.

Ejemplos prácticos sobre demodb:

pontIA

```sql
-- 1. Actualizar el importe de una reserva específica
UPDATE bookings.bookings bk
SET total_amount = 650.00
WHERE   bk.book_ref = 'XYZ123';
-- Validamos que la reserva se ha actualizado correctamente
SELECT *, 800.00 as total_amount_corrected
FROM    bookings.bookings
WHERE   book_ref = 'XYZ123';

-- 2. Corregir el nombre de un pasajero en su billete
UPDATE bookings.tickets tk
SET passenger_name = 'Jonathan Doe'
WHERE   tk.ticket_no = '0005437629810';
-- Validamos que el nombre del pasajero se ha actualizado correctamente
SELECT *,
        'James Rush' as passenger_name_corrected
FROM    bookings.tickets
WHERE   ticket_no = '0005437629810';

-- 3. Actualización múltiple: Marcar como 'Delayed' vuelos que no han salido a
su hora
CREATE TEMPORARY TABLE temp_flights
AS
  SELECT 'T' as reg_type
         ,*
  FROM    bookings.flights
  LIMIT 0;

-- Creamos la tabla temporal para almacenar los vuelos retrasados y volver a su
estado original si es necesario
INSERT INTO temp_flights
SELECT 'T',
        fl.*
FROM    bookings.flights fl
WHERE   fl.actual_departure IS NULL
  AND   fl.scheduled_departure < CURRENT_TIMESTAMP
ORDER BY fl.scheduled_departure DESC;
-- Lanzamos la actualización de los vuelos retrasados
UPDATE bookings.flights fl
SET status = 'Delayed'
WHERE   fl.actual_departure IS NULL
  AND   fl.scheduled_departure < CURRENT_TIMESTAMP;
-- Validamos que los vuelos se han actualizado correctamente
SELECT *,'On Time' as status_corrected
FROM    bookings.flights
WHERE   actual_departure IS NULL
  AND   scheduled_departure < CURRENT_TIMESTAMP
ORDER BY scheduled_departure DESC;
-- Ahora podemos revertir los cambios si es necesario
UPDATE bookings.flights fl
SET status = u.status
FROM    (SELECT flight_id, status FROM temp_flights) u
```

pontIA

```
WHERE   fl.flight_id = u.flight_id
  AND   fl.status = 'Delayed';

-- 4. Error de actualización por violación de PK y su solución
SELECT *
FROM    bookings.tickets
WHERE   ticket_no LIKE '00054376298%'
ORDER BY ticket_no DESC
LIMIT 2;
-- Provocará error si el ticket_no '0005437629810' existe en la tabla
UPDATE bookings.tickets tk
SET ticket_no = '0005437629809'
WHERE tk.ticket_no = '0005437629810';
/*
-- Error que obtenemos
llave duplicada viola restricción de unicidad «tickets_pkey»
DETAIL: Ya existe la llave (ticket_no)=(0005437629809).
*/

-- Solución: Asegurarse de actualizar a un valor de Primary Key que no exista
en la tabla, pero esto nos genera un error por violación de la clave foránea
UPDATE bookings.tickets tk
SET ticket_no = '0005437629811'
WHERE tk.ticket_no = '0005437629809';
/*
-- Error que obtenemos
update o delete en «tickets» viola la llave foránea «segments_ticket_no_fkey»
en la tabla «segments»
DETAIL: La llave (ticket_no)=(0005437629809) todavía es referida desde la tabla
«segments».
*/

--------------------------------------------
-- Parámetro ON UPDATE CASCADE --
--------------------------------------------
CREATE TABLE tabla_padre (
    id INT PRIMARY KEY
);

CREATE TABLE tabla_hija (
    id INT PRIMARY KEY,
    padre_id INT REFERENCES tabla_padre(id) ON UPDATE CASCADE
);

INSERT INTO tabla_padre (id) VALUES (1);
INSERT INTO tabla_hija (id, padre_id) VALUES (1, 1);

SELECT * FROM tabla_padre;
SELECT * FROM tabla_hija;

UPDATE tabla_padre SET id = 100 WHERE id = 1;
```

```
SELECT * FROM tabla_padre;
SELECT * FROM tabla_hija;

DROP TABLE tabla_hija;
DROP TABLE tabla_padre;
```

DELETE

Se utiliza este comando para la eliminación de filas (registros) en las tablas.

Esta sentencia deberá ir siempre con una cláusula WHERE, ya que sin un filtro, DELETE elimina todas las filas de la tabla dada, dejándola vacía.

Escenarios de uso:

- Eliminar una reserva cancelada por error (siempre que no tenga billetes asociados o borrando primero los hijos).
- Purgar vuelos históricos muy antiguos que ya no se analizan.

PontIA

Ejemplos prácticos sobre demodb:

pontIA

```sql
-- 1. Eliminar un billete específico
-- INSERT para no alterar los datos originales de la base de datos
INSERT INTO bookings.tickets (ticket_no, book_ref, passenger_id,
passenger_name, outbound)
VALUES ('1234567890199', 'XYZ123','US 1903569888099', 'John Doe', 'True');
-- Validamos que el billete se ha insertado correctamente
SELECT *
FROM    bookings.tickets
WHERE   ticket_no = '1234567890199';
-- Eliminamos el billete específico
DELETE FROM bookings.tickets tk
WHERE tk.ticket_no = '1234567890199';
-- Validamos que el billete se ha eliminado correctamente
SELECT *
FROM    bookings.tickets
WHERE   ticket_no = '1234567890199';

-- 2. Eliminar múltiples billetes asociados a una misma reserva
-- INSERT para no alterar los datos originales de la base de datos
INSERT INTO bookings.tickets (ticket_no, book_ref, passenger_id,
passenger_name, outbound)
VALUES ('1234567890199', 'XYZ123','US 1903569888099', 'John Doe', 'True')
,       ('1234567890200', 'XYZ123','US 1903569888100', 'Jane Doe', 'True')
,       ('1234567890201', 'XYZ123','US 1903569888101', 'Jim Doe', 'True');
-- Validamos que el billete se ha insertado correctamente
SELECT *
FROM    bookings.tickets
WHERE   book_ref = 'XYZ123';
-- Eliminamos la reserva específica
DELETE FROM bookings.tickets tk
WHERE tk.book_ref = 'XYZ123';

-- Validamos que la reserva se ha eliminado correctamente junto con sus
billetes asociados
SELECT *
FROM    bookings.tickets
WHERE   book_ref = 'XYZ123';

-- 3. Eliminar la reserva
-- INSERT para no alterar los datos originales de la tabla bookings
INSERT INTO bookings.bookings (book_ref, book_date, total_amount)
VALUES ('XYZ124', CURRENT_DATE, 500.00)
-- INSERT para no alterar los datos originales de la tabla tickets
INSERT INTO bookings.tickets (ticket_no, book_ref, passenger_id,
passenger_name, outbound)
VALUES ('1234567890199', 'XYZ124','US 1903569888099', 'John Doe', 'True')
,       ('1234567890200', 'XYZ124','US 1903569888100', 'Jane Doe', 'True')
,       ('1234567890201', 'XYZ124','US 1903569888101', 'Jim Doe', 'True');
-- Validamos que la reserva se ha insertado correctamente en bookings
SELECT *
FROM    bookings.bookings
WHERE   book_ref = 'XYZ124';
```

```sql
-- Validamos que los billetes se han insertado correctamente en tickets
SELECT *
FROM    bookings.tickets
WHERE   book_ref = 'XYZ124';
-- Eliminamos la reserva específica
DELETE FROM bookings.bookings bk
WHERE bk.book_ref = 'XYZ124';
-- Eliminamos los billetes asociados a la reserva eliminada
DELETE FROM bookings.tickets bk
WHERE bk.book_ref = 'XYZ124';
-- Validamos que la reserva se ha insertado correctamente en bookings
SELECT *
FROM    bookings.bookings
WHERE   book_ref = 'XYZ124';
-- Validamos que los billetes se han insertado correctamente en tickets
SELECT *
FROM    bookings.tickets
WHERE   book_ref = 'XYZ124';

-- 4. Eliminar usando condiciones compuestas: Vuelos cancelados del año pasado
-- INSERT para no alterar los datos originales de la tabla flights
INSERT INTO bookings.flights (route_no, status, scheduled_departure,
scheduled_arrival)
VALUES ('PG123', 'Cancelled','2024-12-15 14:25:00', '2024-12-15 18:35:00')
,       ('PG456', 'Cancelled','2024-11-20 02:35:00', '2024-11-20 06:45:00')
,       ('PG789', 'Cancelled','2024-10-05 08:45:00', '2024-10-05 12:55:00');
-- Validamos que los vuelos se han insertado correctamente en flights
SELECT *
FROM    bookings.flights fl
WHERE   fl.status = 'Cancelled'
  AND   fl.scheduled_departure < '2025-01-01';

-- Eliminamos los vuelos cancelados del año pasado
DELETE FROM bookings.flights fl
WHERE   fl.status = 'Cancelled'
  AND   fl.scheduled_departure < '2025-01-01';
  -- Validamos que los vuelos se han eliminado correctamente en flights
SELECT *
FROM    bookings.flights fl
WHERE   fl.status = 'Cancelled'
  AND   fl.scheduled_departure < '2025-01-01';
```

PontIA

Comparativo: SQL vs. NoSQL en Operaciones CRUD

Basado en el análisis de arquitectura de datos, estas son las diferencias críticas para la toma de decisiones:

Característica	PostgreSQL (SQL)	NoSQL (MongoDB/Cassandra)
Modelo de Datos	Relacional (Tablas, filas, columnas)	Múltiples (Documentos, Clave-Valor, Grafos)
Esquema	Estricto y predefinido	Dinámico y flexible
Escalabilidad	Vertical (Aumentar CPU/RAM)	Horizontal (Añadir nodos/servidores)
Garantías	Propiedades ACID (Alta fiabilidad)	Teorema CAP (Eventual Consistency)
Interfaz (API)	Lenguaje SQL Estructurado	APIs basadas en objetos
Rendimiento	Atado a optimización de disco e índices	Optimizado para baja latencia de red

PontIA